

# C på fyra gånger

---

Ulf Bilting [bilting@isk.kth.se](mailto:bilting@isk.kth.se)

Institutionen för Tillämpad IT, KTH

# Kursens syfte

---

- ▶ Kunna skriva enkla program
- ▶ Kunna använda bibliotek
- ▶ Kunna läsa, hitta fel och ändra i C-program

# Uppläggning

---

- Introduktion
  - direkt på programexempel
- Mer metodisk genomgång
  - Variabler, typer, funktioner
  - Pekare och fält
  - Datastrukturer
  - Bibliotek

# Första C-programmet

---

```
#include <stdio.h>

int main()
{
    printf("Hej hopp!\n");
    return 0;
}
```

- Ett C-program är en eller flera filer med C-kod
  - koden är uppdelad i *funktioner*
  - en heter `main`, där startar programkörningen

# Kompilatorn

---

- C-kod förstås inte direkt av processorn
  - måste översättas till maskinkod med en C-kompilator
  - den är ett program som läser C-kod från filer
- Kompilatorn `cc` är ett UNIX-kommando
  - producerar filen `a.out` som innehåller den körbara maskinkoden

```
$ cc 11.c
```

```
$ ./a.out
```

```
Hej, hopp!
```

```
$
```

# Några detaljer

---

- `/* kommentarer utförs inte */`
- För att in/utmatning skall fungera behövs deklARATIONER från filen `stdio.h`
- Alla satser avslutas med semikolon `;`
- `printf` är en funktion för textutmatning
- `return` ger ett värde till anroparen av funktionen
  - Noll från `main` betyder att allt gick bra
- Alla funktioner har huvud och kropp
  - kroppen omsluts av `{ }`
- Ny-rads-tecken skrivs `\n`

# Variabler

---

- ▶ Variabler är ”lådor” i minnet som kan innehålla värden av en viss typ, till exempel
  - heltal heter `int` (integer)
  - flyttal (tal med decimaler) heter `float`
- ▶ Alla variabler måste ges namn, *definieras*, innan de används:

```
int summa, antal, max;  
float vikt, genomsnitt;
```

# Tilldelning och aritmetik

---

- Variabler kan tilldelas värden
- Variablers värden samt konstanter kan användas i uttryck för beräkningar

```
int i, j;
```

```
float p;
```

```
i = 45;
```

```
j = 56 * i;
```

```
p = 4.89 / j + 13;
```



# Exempel, variabler och beräkningar

---

```
#include <stdio.h>

int main()                /* Tidsberäkning */
{
    float tid1, tid2;
    printf("Tid i första åket? ");
    scanf("%f", &tid1);
    printf("Tid i andra åket? ");
    scanf("%f", &tid2);
    printf("Total tid: %f\n", tid1+tid2);
    printf("Genomsnittlig tid: %f\n", (tid1+tid2)/2);
}
```

# Några detaljer

---

## ► Att anropa en funktion:

- `scanf` och `printf` är funktioner
- anrop sker med parenteser (     )
- inom parenteserna kan *parametervärden* skickas med
- Och-tecken, `&`, betyder att variabeln kan ändras av funktionen
- `scanf` och `printf` vill ha en *formatsträng* med *formatkoder* som första parameter
- Heltal `%d`     flyttal `%f`

# Repetition

---

- Ofta skall något utföras flera gånger
  - ett bestämt antal gånger: `for`
  - så länge ett villkor är uppfyllt: `while`

```
int i, j = 1;  /* variabler kan initieras */
for (i = 0; i < 10; i++)
    printf("%d ", i * 12);
while ( j < 10000)
{
    printf("%d ", j);
    j = j * 2;
}
```

# Några detaljer

---

- ▶ Både `for` och `while` vill ha ett *villkor*, som avgör om snurran skall ta ytterligare ett varv
- ▶ Öka-på-med-ett är vanligt: `i++`
- ▶ Det är bara *en* sats som repeteras i `for` och `while`
- ▶ Flera satser görs till *en* sats med klamrar `{ }`

# Exempel, for-sats

---

```
#include <stdio.h>
#define ANTAL_AR 10
#define RANTA 8.5
int main()          /* Ränteberäkning, fast ränta */
{
    float kap;
    int   ar;
    printf("Insatt kapital? "); scanf("%f", &kap);
    printf("\n År          Saldo\n ==          =====\n");
    for (ar = 1; ar <= ANTAL_AR; ar++) {
        kap = kap * (1 + RANTA / 100);
        printf("%3d%13.2f\n", ar, kap);
    }
}
```

# Några detaljer

---

- Konstanter kan uttryckas med `#define`
  - OBS! Define-makron utförs av en ”dum” makroprocessor (textutbytare)
  - Inget semikolon efter `#define`
- Formatkoderna kan ange vidd
  - Kolumnvidd `%3d`
  - Kolumnvidd och antal decimaler `%13.2f`

# Exempel, while-sats

---

```
#include <stdio.h>
#define RANTA 8.5
#define ONSKAT_BELOPP 100000
int main()          /* Beräkning av hur länge */
{                  /* kapitalet måste stå inne */
    float kap;
    int ar;
    printf("Insatt kapital? "); scanf("%f", &kap);
    ar = 0;
    while (kap < ONSKAT_BELOPP) {
        ar = ar + 1;
        kap = kap * (1 + RANTA / 100);
    }
    printf("Kapitalet måste stå inne %d år\n", ar);
}
```

# Villkor

---

- Ett program kan behöva gå olika vägar på grund av något eller några villkor

```
int i, j;
scanf("%d %d", &i, &j);
if (i == 1)
    printf("i är ett!\n");
if (j > 100)
    printf("j är för stort\n")
else
    printf("j har värdet %d\n");
```



# Exempel, villkor

---

```
#include <stdio.h>
int main()          /* Beräkna medelvärde och max-värde */
{
    float tal, sum, storsta;
    int    ant;
    sum = 0;
    storsta = 0;
    ant = 0;
    printf("Skriv in talen.\n");
    printf("Avsluta med END OF FILE.\n");
    while (scanf("%f", &tal) == 1) {
        ant++;
        sum += tal;          /* samma sak som s = s + tal */
        if (tal > storsta)
            storsta = tal;
    }
    printf("Medelvärdet:    %f\n", sum / ant);
    printf("Största talet: %f\n", storsta);
}
```

# Egna C-funktioner

---

- Att skriva egna C-funktioner har flera syften
  - att slippa upprepa kod på många ställen
  - att sätta ett bra namn på något så att man slipper tänka på alla detaljer varje gång
  - att paketera kod så att andra kan återanvända den
  - att skapa läsbarhet, ändringsbarhet och flyttbarhet (portabilitet)

# Exempel, C-funktion

---

- En funktion kan ha många parametrar men bara *ett* returvärde

```
float f_to_c( float fahrenheit )  
{  
    float celsius;  
    celsius = (fahr - 32) * 5 / 9;  
    return celsius;  
}
```

# Exempel, C-funktion

---

- En funktion kan vara utan parametrar och/eller returvärde

```
void pip()  
{  
    int i;  
    for (i = 0; i < 100; i++)  
        printf("pip \a ");  
    /* \a betyder "alert-teckenkod"  
       dvs piper eller plingar */  
}
```

# Exempel, C-funktion

---

```
#include <stdio.h>
float max(float x, float y)    /* ger max-värde */
{
    if (x > y)
        return (x);
    else
        return (y);
}

int main()                    /* Beräkna medelvärde och max-värde */
{
    float tal, sum, storsta;
    int  ant;
    sum = 0;
    storsta = 0;
    ant = 0;
    printf("Skriv in talen.\n");
    printf("Avsluta med END OF FILE.\n");
    while (scanf("%f", &tal) == 1) {
        ant++;
        sum += tal;
        storsta = max(storsta, tal);
    }
    printf("Medelvärdet:   %f\n", sum / ant);
    printf("Största talet: %f\n", storsta);
}
```

# Fält

---

- ▶ Ibland vill man ha många av en sort
  - t ex tio flyttal: `float f[10];`
- ▶ *Elementen* väljs ut med *index* som är heltal
  - elementens index går från 0 till 9 !
  - elementen används precis som vanliga flyttal

```
int i = 6;  
float f[10];  
f[0] = 3.4;  f[i + 1] = 5.4  
f[i + 4] = 5.6    /* FEL som inte kollas! */  
printf("%f", f[7]);
```

# Tecken: värden och variabler

---

- Tecken representeras i datorer med koder, som är heltal
- C kallar dessa heltal `char` (av character)
- Teckenkoder noteras med enkla citattecken: `'w'`

```
char c;  
int i;  
c = 'A';  
i = c;  
printf("%c  %d", c, i);
```

# Textsträngar är teckenfält

---

## ► Text lagras i teckenfält

- fältet kan vara större än vad som för tillfället behövs
- slut på strängen markeras med specialkod `'\0'` som är heltalet noll
- Strängen `"Ulf"` ser alltså ut som

|     |     |     |      |
|-----|-----|-----|------|
| 'U' | 'l' | 'f' | '\0' |
|-----|-----|-----|------|



# Exempel, textsträngar

---

```
#include <stdio.h>  /* filen 26.c */

int main()          /* Läsning och skrivning av namn */
{
    char namn[20];
    printf("Vad heter du?\n");
    scanf("%s", namn);
    printf("Hej %s\n", namn);
}
```

# Några detaljer

---

- Inläsning av textsträng kan ske med `scanf`
  - formatkoden är `%s`
  - inget `&` före fältnamnet (namn är redan en *pekare*, vi återkommer!)
  - en följd av tecken mellan ”vitt” anser `scanf` vara en textsträng
  - vi *hoppas* att fältet räcker till för den inlästa strängen

# getchar och putchar

---

- ▶ Läser och skriver enstaka tecken
  - getchar hoppar inte över ”vita” tecken
  - getchar returnerar EOF vid slut på indata
  - indata *buffras* av UNIX, så getchar får inte tag i något tecken förrän <retur>-tangenter trycks ned
  - putchar skriver ut *ett* tecken

# Exempel, getchar

---

```
#include <stdio.h>    /* filen 28.c */

int main()             /* Beräkning av antal siffror i en text */
{
    int c, ant;
    ant = 0;
    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9')
            ant++;
    }
    printf("Antal siffror: %d\n", ant);
}
```

# Några detaljer

---

- Tilldelningar har ett värde
  - Tilldelningen `c = getchar()` ger det inlästa tecknet som värde
  - värdet kan jämföras med `EOF`, notera parenteser!
  - Icke lika med uttrycks med `!=`
- Villkor kan kombineras med `&&` och `||`
  - båda sidorna måste vara sanna för `&&`
  - minst ena sidan sann för `||`

# Exempel, läsning av hel rad

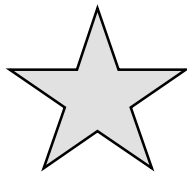
---

```
#include <stdio.h>
int main()    /* Läsning och skrivning av hel rad */
{
    char namn[20], c;
    int i;
    printf("Vad heter du?\n");
    i = 0;
    while ((c = getchar()) != '\n') {
        namn[i] = c;
        i++;
    }
    namn[i] = '\0';
    printf("Hej %s\n", namn);
}
```

# Komplettera exemplet

---

- Kontrollera att fältet räcker
- Kontrollera att det inte är slut på indata



- Slut på genomgången av C
- Alla huvudsaker genomgångna, många detaljer kvar